



A First Taste of AI.

Today your apps start feeling smart. TF-IDF, text similarity, and the exact technique behind resume matchers, plagiarism checkers, and search engines. No neural networks — just good math.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

13 of 25

How we'll spend our 90 minutes today.

Text similarity, TF-IDF, cosine similarity, scikit-learn, and a resume-vs-job matcher. The AI technique behind real tools — in 90 minutes.

00:00-00:10

01

Day 12 Recap & Q&A

APIs, requests, JSON, caching, the weather dashboard. Quick check.

00:10-00:25

02

Text Similarity Intuition

How do you measure how alike two texts are? The mental model before any math.

00:25-00:40

03

TF-IDF Explained Simply

Term Frequency $\times$ Inverse Document Frequency. No heavy math — plain language.

00:40-00:55

04

Cosine Similarity + scikit-learn

The score from 0 to 1. TfidfVectorizer + cosine_similarity. 5 lines of code.

00:55-01:05

05

How Real AI Uses This

Search engines, resume matchers, plagiarism checkers, recommendation systems.

01:05-01:30

06

Exercise: Resume Matcher

Upload a resume, paste a job description, get a match score + missing keywords. The exact technique from the Proposal.

Yesterday your apps reached the internet. Today they start thinking.

Quick check that Day 12's weather dashboard runs. Today's exercise doesn't call an API — but the group project's resume matcher will combine Day 11 (PDF extraction) + Day 12 (APIs) + Day 13 (TF-IDF).

WHAT WE COVERED YESTERDAY

- ✓ What is an API — the restaurant analogy
- ✓ requests.get + response.json — the 4-line pattern
- ✓ Query parameters — params dict vs manual URL
- ✓ Error handling — timeout, raise_for_status, status codes
- ✓ APIs in Streamlit — @st.cache_data + the run-loop gotcha

CHECK YOUR LAPTOP NOW

```
# You should have Day 12's weather app:  
$ ls day12/  
weather_dashboard.py  
  
# Run it — should show live weather:  
$ streamlit run weather_dashboard.py  
# → Pick a city, see live temperature
```

IF THE APP DOESN'T RUN — check: (1) requests is installed, (2) internet is on, (3) the API URL is correct. Today uses scikit-learn — `pip install scikit-learn`.

The technique behind 4 of your 10 project ideas.

TF-IDF + cosine similarity is not a toy — it powers search engines, resume matchers, plagiarism checkers, and recommendation systems. Today you learn the foundation.

GROUP PROJECTS THAT USE THIS

- 1 Resume vs Job Description Matcher**
Upload a resume, compare to a job description, get a match score. TF-IDF + cosine similarity. THE Day 13 exercise.
- 2 Plagiarism & Similarity Checker**
Compare two documents, get a similarity score. Same TF-IDF technique, different use case.
- 3 Smart Notes Summarizer**
Score sentences by importance. TF-IDF finds the key words, the sentences with them are the summary.
- 4 Campus FAQ Assistant**
Match a student question to stored answers. TF-IDF + cosine similarity finds the closest match.

REAL-WORLD AI TOOLS

- Search engines**
Google ranks results by text similarity to your query. TF-IDF was the foundation — modern search evolved from it.
- ATS (Applicant Tracking Systems)**
Companies screen resumes with TF-IDF. Your resume is scored against the job description. Same technique you build today.
- Plagiarism checkers**
Turnitin compares your essay to millions of documents. Similarity score = cosine similarity.
- Recommendation systems**
"Similar items" on Amazon/Netflix. Not just text — but the same mathematical idea: measure similarity between vectors.

THE EVOLUTION

TF-IDF (today) → word embeddings (Word2Vec, GloVe) → transformer embeddings (BERT, GPT).

Each generation is smarter, but the core idea is the same: represent text as numbers, measure the distance. Today you learn step 1.

How alike are two pieces of text? That's the whole question.

Before any math, let's think about what "similar" means. Two texts are similar if they talk about the same things, using similar words. Computers can't "read" — so we turn text into numbers, then compare the numbers.

WHAT DOES SIMILAR MEAN?

Text A: "Python is a programming language for data science."

Text B: "Python is used for data analysis and machine learning."

0.72  HIGH — Python + data

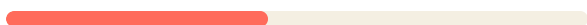
Text A: "Python is a programming language for data science."

Text C: "The weather in Amritsar is 28 degrees today."

0.03  LOW — no shared words

Text A: "Python is a programming language for data science."

Text D: "Java is a programming language for web development."

0.45  MED — shares "programming language"

THE 3-STEP PROCESS

1 Turn text into numbers
Count the words. Weight them by importance. Each text becomes a vector (a list of numbers).

2 Compare the numbers
Measure how "close" the two vectors are. Cosine similarity gives a score from 0 (unrelated) to 1 (identical).

3 Return the score
0.72 = quite similar. 0.03 = unrelated. 1.0 = identical text. The score is your answer.

This is what TF-IDF + cosine similarity does. TF-IDF turns text into vectors. Cosine similarity compares them. Two steps, one score.

KEY IDEA Computers don't read text. They read numbers. TF-IDF is the translation layer — text in, numbers out. Cosine similarity is the comparison — numbers in, score out.

TF-IDF — which words matter most?

Term Frequency × Inverse Document Frequency. A fancy name for a simple idea: a word is important if it appears often in ONE document, but rarely across ALL documents. **“the”** is common everywhere = not important. **“scikit-learn”** is rare = very important.

TERM FREQUENCY (TF)

HOW OFTEN?

Count how many times a word appears in a document, divided by total words.

Example: in the text **“Python Python data science”**, “Python” appears 2 of 4 words → **TF = 0.5**

A word that appears more often gets a higher TF score.

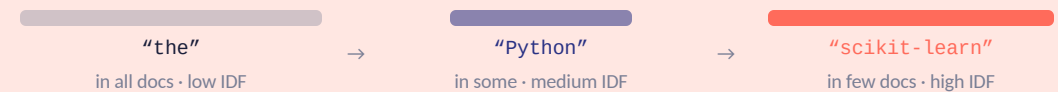
Python		2
data		1
science		1

INVERSE DOCUMENT FREQUENCY (IDF)

HOW RARE?

If a word appears in EVERY document (like “the”, “is”, “and”), it’s not distinctive. IDF penalizes common words.

Example: **“the”** appears in all 100 docs → **IDF = $\log(100/100) = 0$** (zero weight). **“scikit-learn”** appears in 2 of 100 → **IDF = $\log(100/2) = 3.9$** (high weight).

**TF × IDF = THE WEIGHT****TF × IDF = weight**

Multiply the two scores. A word that appears often (high TF) AND is rare across documents (high IDF) gets the highest weight — that’s the word that matters most.
Example: “Python” in a resume — appears often (high TF) + not in every document (high IDF) = high TF-IDF score. That word defines this document.

💡 **INTUITION** — TF-IDF finds the words that make each document **UNIQUE**. “the” is everywhere = weight 0. “Python” is distinctive = high weight. These weighted words become the document’s fingerprint.

Cosine similarity — how close are two vectors?

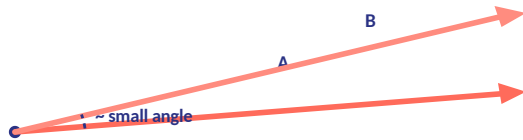
TF-IDF turns each text into a vector (a list of numbers). Cosine similarity measures the **ANGLE** between two vectors. **Small angle = similar. Large angle = different.** The score goes from **0** (unrelated) to **1** (identical).

THE VISUAL

ANGLE = MATCH

SIMILAR TEXTS

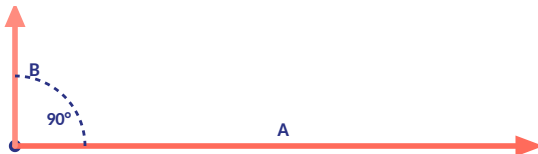
score = 0.85



Small angle = similar texts. The vectors point in almost the same direction.

UNRELATED TEXTS

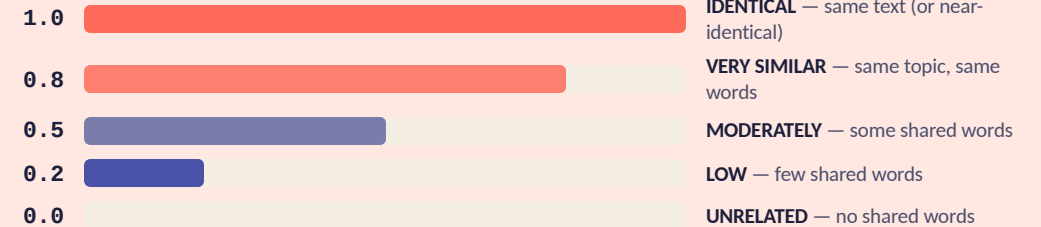
score = 0.00



90° angle = unrelated texts. The vectors point in completely different directions.

THE SCORE SCALE

0 → 1



For resume matching: 0.7+ = strong match. 0.4–0.7 = partial. Below 0.4 = weak. Threshold depends on use case.

```
from sklearn.metrics.pairwise import cosine_similarity
score = cosine_similarity([vector_a], [vector_b])[0][0] # → 0.72 (a float between 0 and 1)
```

💡 **INTUITION** — Don't worry about the trigonometry. Just remember: cosine similarity = how close the arrows are. Close = similar. Far = different. The number between 0 and 1 is your match score.

5 lines of code. That's the whole technique.

scikit-learn's **TfidfVectorizer** turns text into TF-IDF vectors. **cosine_similarity** compares them. You write 5 lines, the library does all the math. **This is the pattern behind every text-similarity app.**

 tfidf_demo.py

python • sklearn

```
# pip install scikit-learn
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# The two texts to compare
resume = "Python developer with experience in data science, machine learning, and SQL."
job_desc = "Looking for a Python engineer skilled in data analysis, ML, and databases."

# Step 1: Create the vectorizer
vectorizer = TfidfVectorizer(stop_words='english')

# Step 2: Transform texts into TF-IDF vectors
vectors = vectorizer.fit_transform([resume, job_desc])

# Step 3: Compute cosine similarity
score = cosine_similarity(vectors[0], vectors[1])[0][0]

print(f"Match score: {score:.2f}")
# → Match score: 0.72

# Step 4 (bonus): Find which words drove the score
feature_names = vectorizer.get_feature_names_out()
# → ['data', 'databases', 'engineer', 'learning', ...
#    'python', 'science', 'sql']
```

THE 5 STEPS

CALL → USE

- 1 **TfidfVectorizer()**
Creates the tool that converts text to TF-IDF vectors. `stop_words="english"` removes common words (the, is, a).
- 2 **fit_transform([text1, text2])**
Learns the vocabulary from both texts, then converts each to a vector. Returns a sparse matrix.
- 3 **cosine_similarity(v1, v2)**
Computes the cosine of the angle between the two vectors. Returns a 2D array — take `[0][0]` for the score.
- 4 **score**
A float between 0 and 1. 0 = unrelated, 1 = identical. **0.72 = strong partial match.**
- 5 **get_feature_names_out()**
Bonus: lists the words the vectorizer found. Useful for showing which keywords matched (or are missing).

⇒ CROSS-REF: Day 6 — Like pandas doing the heavy lifting for data, scikit-learn does the heavy lifting for math. You call the function, the library does the computation. Same pattern: import → call → use the result.

TF-IDF is only as good as the text you feed it.

The vectorizer does **SOME** cleaning automatically (lowercasing, basic punctuation). But for real-world text — resumes, job descriptions, messy PDFs — you need to clean first. **This connects to Day 11's regex skills.**

WHAT THE VECTORIZER DOES

AUTO



Auto-lowercases

"Python" and "python" are treated as the SAME word. You don't need to lowercase manually — but KNOW this, so you don't get confused when they match.



Strips basic punctuation

"Python," becomes "python". Commas, periods, exclamation marks removed automatically.



Removes stop words (if you ask)

stop_words='english' removes "the", "is", "a", "and" — common words with no meaning. **ALWAYS use this.**



fit vs transform — the production pattern

```
# EXERCISE PATTERN (both texts at once):
vectors = vectorizer.fit_transform([resume, job_desc])

# PRODUCTION PATTERN (fit once, transform many):
vectorizer.fit([job_desc]) # learn vocab
resume_vector = vectorizer.transform([resume1])
resume_vector = vectorizer.transform([resume2])
# Same vectorizer, different resumes — faster
```

WHAT YOU SHOULD CLEAN FIRST

MANUAL



Strip extra whitespace

```
re.sub(r'\s+', ' ', text)
```

PDFs and Word docs have weird spacing. Multiple spaces, tabs, newlines → normalize to single spaces.



Remove URLs and emails

```
re.sub(r'\S+@\S+', '', text)
```

Email addresses add noise to TF-IDF. Remove them before vectorizing.



Remove special characters

```
re.sub(r'^a-zA-Z0-9\s]', '', text)
```

Keep only letters, digits, spaces. Remove ©, ™, bullet points, etc.



Remove numbers (optional)

```
re.sub(r'\d+', '', text)
```

Phone numbers, years, IDs add noise — but keep numbers if they matter (e.g., "Python 3" vs "Python 2").

REAL-WORLD EXAMPLE — Clean a resume before TF-IDF

python • day 11 + day 13

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer

raw_text = "Python Developer\n\nSkills: Python, SQL, pandas\nContact: aarav@email.com, +91 98765 43210"

# Step 1: Clean (Day 11 regex skills)
cleaned = re.sub(r'\S+@\S+', '', raw_text) # remove email
cleaned = re.sub(r'\+?\d{8,}\d', '', cleaned) # remove phone
cleaned = re.sub(r'\s+', ' ', cleaned) # normalize whitespace
cleaned = re.sub(r'^a-zA-Z0-9\s]', '', cleaned) # remove special chars
# → "Python Developer Skills Python SQL pandas Contact"
```

```
DAY 13 • A FIRST TASTE OF AI
# Step 2: Vectorize (Day 13 TF-IDF)
vectorizer = TfidfVectorizer(stop_words='english')
```

TF-IDF is powerful. But it has blind spots.

TF-IDF treats words as independent tokens. It doesn't understand meaning, relationships, or context. **These limitations are WHY Word2Vec, BERT, and GPT were invented.** Understanding the gaps helps you know when TF-IDF is enough — and when you need something smarter.

→ SYNONYMS

LIMITATION 01

TF-IDF treats **"ML"** and **"machine learning"** as COMPLETELY DIFFERENT words. They get separate TF-IDF scores. A resume saying "ML" won't match a job saying "machine learning" — even though they mean the same thing.

```
Resume: "3 years of ML experience"
Job:    "Machine learning engineer wanted"
Score:  0.12 ← LOW! TF-IDF doesn't know ML = machine learning
```

NEXT GEN: Word2Vec learns that "ML" and "machine learning" appear in similar contexts → similar vectors → high similarity.

◇ CONTEXT

LIMITATION 03

TF-IDF treats **"bank"** the same whether you mean "river bank" or "State Bank of India". The word has one TF-IDF score regardless of context. It can't tell the difference.

```
Text A: "I sat by the river bank"
Text B: "I deposited money in the bank"
Score:  0.34 ← MEDIUM! "bank" matches, but the MEANING is different
```

NEXT GEN: BERT generates different vectors for "bank" based on surrounding words. Same word, different vector depending on context.

✱ WORD RELATIONSHIPS

LIMITATION 02

TF-IDF treats **"developer"** and **"development"** as unrelated words. They share a root but TF-IDF doesn't know that. Stemming (cutting to "develop") helps, but TF-IDF alone doesn't do it.

```
Resume: "Python developer, 3 years"
Job:    "Looking for software development experience"
Score:  0.08 ← LOW! "developer" ≠ "development" to TF-IDF
```

NEXT GEN: BERT understands that "developer" and "development" share a root → related vectors.

≡ WORD ORDER

LIMITATION 04

TF-IDF ignores word order. **"dog bites man"** and **"man bites dog"** get the SAME score — the words are the same, just in different order. But the meaning is opposite.

```
Text A: "The dog bites the man"
Text B: "The man bites the dog"
Score:  1.0 ← IDENTICAL! TF-IDF ignores word order
```

NEXT GEN: Transformers (BERT, GPT) process words in sequence → different vectors for different word orders.

💡 **THE TAKEAWAY** — TF-IDF is great for: keyword matching, document similarity, quick prototyping. It struggles with: synonyms, word relationships, context, word order. For the group project, TF-IDF is ENOUGH. For production AI, upgrade to embeddings (BERT/GPT). Know the limits, choose the right tool.

TF-IDF is step 1 of a 3-step evolution. It's where modern AI started.

The technique you learn today isn't obsolete — it's the **foundation**. Search engines, ATS systems, and ChatGPT all evolved from this exact idea: **represent text as numbers, measure similarity**.

YOU ARE HERE • DAY 13

TF-IDF

(1970s – 2010s)

- Count words
- Weight by rarity
- Compare angles
- **Score: 0-1**

You are here — Day 13

Word Embeddings

(2013 – 2018)

- **Word2Vec**
- **GloVe**
- Meaning-based
- **King – Man + Woman = Queen**
- 300-dim vectors

Better, not different idea

Transformer Embeddings

(2018 – today)

- **BERT**
- **GPT**
- Context-aware
- Same idea: text → vector → similarity
- 768+ dim

ChatGPT uses this with 175B parameters

🔍 SEARCH ENGINES

Google's early algorithm (PageRank + TF-IDF) ranked pages by how similar their text was to your query. Modern search uses neural networks, but the core idea — **measure text similarity** — is unchanged.

📄 ATS (APPLICANT TRACKING)

When you submit a resume online, an ATS scores it against the job description using **TF-IDF or embeddings**. Same technique you build today. Companies like Workday, Greenhouse, Lever all use this.

💬 CHATGPT

ChatGPT represents text as **768+ dimensional vectors** (vs TF-IDF's word-count vectors). But the core idea is the same: text → numbers → measure distance. ChatGPT is smarter, not fundamentally different.

🚩 **THE TAKEAWAY** — You're not learning a toy technique. You're learning the foundation that every modern AI tool built on. **TF-IDF → Word2Vec → BERT → GPT**. Same idea, increasing sophistication. Today is step 1 — and it's the most important step.

• PART 3 OF 3 · CLOSING EXERCISE

03 / 03

Build a resume matcher.

Upload a resume, paste a job description, get a match score + missing keywords. The exact technique from the Proposal — and the same technique behind real ATS systems.

BRIEF (5 MIN)

BUILD (20 MIN)

EXTEND (5 MIN)

Resume vs job description. Match score + missing keywords.

Build a Python script that takes a resume text and a job description text, computes a match score using TF-IDF + cosine similarity, and lists the keywords the resume is missing.

THE BRIEF

Build a script called `resume_matcher.py` that compares a resume to a job description and outputs: (1) a match score from 0 to 1, (2) a list of keywords from the job description that are missing from the resume.

REQUIREMENTS

- ✓ Use `TfidfVectorizer` with `stop_words='english'`
- ✓ Use `cosine_similarity` to compute the match score
- ✓ Extract keywords from the job description (high TF-IDF words)
- ✓ Check which of those keywords are NOT in the resume
- ✓ Print: match score + missing keywords list

TASK

STARTING SCAFFOLD

SCAFFOLD

```
resume_matcher.py

# resume_matcher.py - starting scaffold
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Sample data
resume = """Python developer with 3 years experience.
Skills: Python, SQL, pandas, Streamlit, Git.
Built data dashboards and REST APIs."""

job_desc = """Looking for a Python engineer with
experience in machine learning, scikit-learn,
data analysis, SQL, and AWS."""

# TODO: Create TfidfVectorizer(stop_words='english')
# TODO: vectorizer.fit_transform([resume, job_desc])
# TODO: cosine_similarity(vectors[0], vectors[1])[0][0]
# TODO: Print match score

# BONUS: Find missing keywords
```

RUN IT — `python resume_matcher.py`. Should print: match score (~0.4) + missing keywords (machine, learning, scikit, aws). The technique behind real ATS systems.

Every student has a working resume matcher. The exact technique behind real ATS systems and 1 of the 10 group project ideas.

```
# Check if it appears in the resume
# TODO: Print missing keywords
```

One way to solve it. There are others — this is just clean.

Walk through this line by line. Then go back to your own version and compare.

SOLUTION

resume_matcher.py

resume_matcher.py

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
```

① resume = """Python developer with 3 years experience.
Skills: Python, SQL, pandas, Streamlit, Git.
Built data dashboards and REST APIs."""
job_desc = """Looking for a Python engineer with
experience in machine learning, scikit-learn,
data analysis, SQL, and AWS."""

② vectorizer = TfidfVectorizer(stop_words='english')
vectors = vectorizer.fit_transform([resume, job_desc])

③ score = cosine_similarity(vectors[0], vectors[1])[0][0]
print(f"Match score: {score:.2f}")
→ Match score: 0.42

④ feature_names = vectorizer.get_feature_names_out()
job_vector = vectors[1].toarray()[0]

⑤ # Find keywords in job_desc with high TF-IDF
important = []
for i, weight in enumerate(job_vector):
 if weight > 0.1: # threshold
 word = feature_names[i]
 # Check if this word is in the resume
 if word not in resume.lower():
 important.append((word, weight))

LINE BY LINE

ANNOTATE

- ① **Sample texts**
Hardcoded resume and job description. In the group project, you'll extract the resume from a PDF (Day 11).
- ② **Vectorize**
TfidfVectorizer with stop_words removes common words. fit_transform learns vocab + creates vectors.
- ③ **Match score**
cosine_similarity returns a 2D array. [0][0] is the score. 0.42 = partial match.
- ④ **Feature names**
get_feature_names_out() lists all words the vectorizer found. job_vector is the job desc's TF-IDF weights.
- ⑤ **Find missing keywords**
For each word with high TF-IDF in the job desc, check if it appears in the resume. If not, it's missing.
- ⑥ **Sort + display**
Sort by importance (highest first). Print the missing keywords with their weights. This is what real ATS systems show.

THE PIPELINE — Resume text → TF-IDF vector → cosine similarity vs job desc → score. Missing keywords → TF-IDF weights → filter by threshold → sort. This is the exact pipeline for the group project.

DAY 13 • A Print(FIRST {word} (importance: {weight:.2f}))
→ Missing keywords:
- machine (0.27)
- python (0.27)

Finished early? Build a Streamlit UI. Or peek at tomorrow.

Three extension ideas for fast students. Plus a preview of Day 14 — the full resume matcher as a class build.

EXTENSION IDEAS

EXTEND • 5 MIN

1

Add a Streamlit UI

Wrap the script in Streamlit. `st.text_area` for the resume + job description, `st.metric` for the score, `st.write` for missing keywords. The group project pattern.

2

Extract resume from PDF

Combine with Day 11: use PyPDF2 to read a resume PDF, then pass the extracted text to the matcher. The full pipeline: PDF → text → TF-IDF → score.

3

Compare multiple resumes

Score 3 resumes against one job description. Sort by score. Show which candidate is the best match. The ATS pattern.

WHAT'S NEXT • DAYS 14-15

AI PART 2 + RECAP

<>

Day 14 — Full resume matcher build

We build the complete app together: PDF extraction + TF-IDF + Streamlit UI. The capstone of Phase 3.

↺

Day 15 — Recap & doubt clearing

Consolidate everything from Days 9-14. Open Q&A. Fix shaky concepts before the group project.

👥

Day 16 — Group project begins

Groups formed, projects picked, sprints start. The technique from today is in 4 of the 10 project ideas.

TOMORROW —

We build the full resume matcher together. PDF extraction (Day 11) + TF-IDF (today) + Streamlit UI (Days 4-5). The capstone before the group project.

IF YOU'RE STUCK — Check: (1) scikit-learn is installed (`pip install scikit-learn`), (2) you passed a LIST of two texts to `fit_transform`, (3) you took `[0][0]` from the `cosine_similarity` result.

Your apps just got smart.

TF-IDF + cosine similarity. The technique behind search engines, ATS systems, and ChatGPT. Tomorrow we build the full resume matcher together.

// session_end

13/25

OPEN THE FLOOR

ASK NOW

✓ Did the TF-IDF intuition click, or does it feel like magic?

✓ Was the cosine similarity score what you expected?

BETWEEN SESSIONS

GET UNSTUCK

✓ Email • hello@zlaark.com

For TF-IDF questions, paste your code + the two texts + the score you got.

✓ Web • www.zlaark.com

Program updates and resources.

WHAT'S NEXT • DAYS 14-15

BUILD + RECAP

✓ Day 14

Full resume matcher build (PDF + TF-IDF + Streamlit).

✓ Day 15

Recap & doubt clearing before the group project.